

Exact Algorithms for Combinatorial Optimization

Fourth Presentation Task

Full Outline

1. Exact Algorithms in Optimization
2. Complete Enumeration (Brute Force)
3. Dynamic Programming
4. Branch-and-Bound
5. Constraint Programming
6. Decomposition Methods

1. Exact Algorithms in Optimization

1. Exact Algorithms in Optimization

- 2. Complete Enumeration (Brute Force)
- 3. Dynamic Programming
- 4. Branch-and-Bound
- 5. Constraint Programming
- 6. Decomposition Methods

Establishing the baseline mathematical requirement for finding provably optimal solutions before exploring specific algorithmic frameworks.

Algorithm Classifications

Exact Algorithms	Heuristic Algorithms	Approximation Algorithms
Always return a provably optimal solution (or correct yes/no answer) when a solution exists; correctly report infeasibility. Guarantee optimality for every instance, often with exponential worst-case complexity on NP-hard problems.	Use problem-specific rules or local search to quickly obtain good solutions. Offer no general worst-case guarantee on closeness to optimality.	Return solutions with a provable performance ratio relative to the optimum, but need not attain the exact optimum.

Bounds and Relaxations

Relaxations:

Replacing hard constraints by easier ones (e.g., LP relaxations of integer programs). Yields a problem whose optimal value is a **bound** on the original problem.

Bounds:

Upper bounds (for maximization) or lower bounds (for minimization). Used in **exact search methods** to prune parts of the search space that cannot contain better solutions than the best incumbent.

2. Complete Enumeration (Brute Force)

1. Exact Algorithms in Optimization
- 2. Complete Enumeration (Brute Force)**
3. Dynamic Programming
4. Branch-and-Bound
5. Constraint Programming
6. Decomposition Methods

Systematically evaluating the worst-case exponential bounds of combinatorial problems to understand the necessity of advanced algorithms.

Complete Enumeration Terminology

- **Complete Enumeration (Brute Force):** The method that systematically generates and evaluates all feasible solutions of a problem, selecting the best one for optimization, or checking all candidates until a required property is found.
- **Computational Complexity:** For NP-hard combinatorial problems, the number of feasible solutions typically grows exponentially with the input size. Requires time exponential in the problem size (e.g., $O(2^n)$ for a 0/1 problem with n binary variables).

Numerical Example: Candidate Enumeration

Problem: Given an Integer Program (IP) with five variables: $x_1, x_2 \in \{0, 1\}$, $x_3 \in \{1, \dots, 10\}$, and $x_4, x_5 \in \{j \in \mathbb{Z} \mid j \text{ is odd}, 0 < j < 20\}$.

Objective: Calculate the exact number of candidate assignments required for a naive complete enumeration over the unrestricted domains.

1. Determine values for x_1 : 2 values.
Determine values for x_2 : 2 values.
2. Determine values for x_2 : 2 values.
3. Determine values for x_3 : 10 values.
4. Determine values for x_4 : Odd integers in $\{1, 3, \dots, 19\}$, yielding 10 values.
5. Determine values for x_5 : Same domain as x_4 , yielding 10 values.
6. Compute product: $2 \times 2 \times 10 \times 10 \times 10 = 4000$ candidate assignments.

3. Dynamic Programming

1. Exact Algorithms in Optimization
2. Complete Enumeration (Brute Force)
- 3. Dynamic Programming**
4. Branch-and-Bound
5. Constraint Programming
6. Decomposition Methods

Overcoming exponential enumeration limits by mathematically decomposing problems and storing overlapping state solutions.

Dynamic Programming Fundamentals

Dynamic Programming (DP): Solves optimization problems by decomposing them into overlapping subproblems, solving each smaller subproblem once, and storing its solution in a table to reuse instead of recomputing.

Overlapping Subproblems: Occurs when the same subproblem, or the same state, is encountered multiple times in the natural recursive formulation.

Principle of Optimality: Bellman's principle states that an optimal policy has the property that, regardless of the initial state and decision, the remaining decisions must constitute an optimal policy with respect to the state resulting from the first decision.

Components of a Dynamic Program

- | | |
|--|--|
| <ul style="list-style-type: none">• States: Variables describing the exact status of the subproblem. | <ul style="list-style-type: none">• Boundary / Initialization: The base cases where evaluation begins. |
| <ul style="list-style-type: none">• Decision Variables: Choices mathematically available at each state. | <ul style="list-style-type: none">• Evaluation Order: An ordering that ensures predecessor states are strictly computed before they are referenced. |
| <ul style="list-style-type: none">• Recurrence (Bellman) Equation: Expresses the mathematical value of a state strictly in terms of smaller states. | <ul style="list-style-type: none">• Reconstruction Rule: The procedure to retrieve an optimal sequence of decisions from the populated table. |

Numerical Example: 0/1 Knapsack DP

Problem: Given item profits $p = (6, 4, 5)$, item weights $w = (3, 2, 4)$, and total knapsack capacity $W = 5$.

Objective: Formulate the Dynamic Programming components to maximize total profit without exceeding weight capacity W .

1. Define States: Subproblem stage i (items considered) and current capacity c .
2. Define Recurrence Equation: $V(i, c)$ evaluates maximum profit possible.
3. Define Worst-Case Complexity: $O(nW)$, dependent on number of items n and capacity W .

Forward vs. Backward Dynamic Programming

Backward DP

References previous stages.
Uses classical $V(i, c)$
recurrence to look back at
solved states.

Forward DP

Focuses exclusively on
reachable states as the
evaluation moves from
earlier to later stages.

Note: Both rely on the identical dynamic programming principle and share the identical worst-case complexity of $O(nW)$.

4. Branch-and-Bound

1. Exact Algorithms in Optimization
2. Complete Enumeration (Brute Force)
3. Dynamic Programming
- 4. Branch-and-Bound**
5. Constraint Programming
6. Decomposition Methods

Using mathematical relaxations to aggressively prune the search space, avoiding exhaustive exploration of non-promising regions.

Branch-and-Bound Terminology

Branch-and-Bound (B&B): An exact tree search algorithm that recursively partitions the feasible region into subproblems and uses bounds from relaxations to discard subproblems.

Branching: Recursively partitioning the feasible region into distinct mathematical subproblems.

Pruning: Stopping exploration of a subproblem and discarding its descendants because the node is either infeasible, already worse than the incumbent best solution, or yields an integer solution that is fully processed.

Numerical Example: Branch-and-Bound Setup

Problem: Maximize $4x_1 + 3x_2$ subject to $2x_1 + x_2 \leq 7$, $x_1 + 2x_2 \leq 7$, with $x_1, x_2 \in \mathbb{Z}_{\geq 0}$.

Objective: Solve the continuous LP relaxation to establish the root node bound before branching on fractional variables.

1. Relax integer constraints to allow fractional variables.
2. Calculate Root Node (Node 0) LP solution: $(7/3, 7/3)$.
3. Determine fractional objective bound: $49/3$.
4. Identify fractional variable for branching: Select x_1 to partition the space.

Numerical Example: Branching Execution

Problem: Partition the fractional Root Node solution $x_1 = 7/3$.

Objective: Branch, bound, and prune subproblems to identify the optimal integer solution.

- **Node 1:** Add constraint $x_1 \leq 2$.

Solve new LP: $\max 4x_1 + 3x_2$ s.t. prior constraints + $0 \leq x_1 \leq 2$.

Resulting intersection yields $x_2 = 2.5$, objective 15.5. (Requires further branching).

- **Node 2:** Add constraint $x_1 \geq 3$.

Yields bounds leading to optimal integer configuration.

- **Optimal Integer Solution:** The best integer solution found is $x_1^* = 3$, $x_2^* = 1$, with objective $z^* = 15$.

5. Constraint Programming

1. Exact Algorithms in Optimization
2. Complete Enumeration (Brute Force)
3. Dynamic Programming
4. Branch-and-Bound
- 5. Constraint Programming**
6. Decomposition Methods

Shifting from algebraic bounds to pure logic paradigms, utilizing systematic search and inference to prune infeasible variable values.

Constraint Programming Terminology

Constraint Programming (CP)

A paradigm stating combinatorial problems as satisfaction/optimization problems over variables with specified domains, utilizing inference to prune values.

Global Constraint

Constraints capturing complex relations between a non-fixed number of variables (e.g., AllDifferent, Cumulative).

Symmetry Breaking

Constraints added to eliminate equivalent permutations of solutions (e.g., lexicographic ordering).

Constraint Propagation

Iteratively narrowing variable domains by enforcing local consistency and removing values that cannot be part of any global solution.

Numerical Example: Task Scheduling Propagation

Problem: Three tasks (A, B, C) must be scheduled exactly once across time slots 1, 2, 3. Task A must precede B ; Task C cannot be in slot 1.

Objective: Use constraint propagation to reduce domains and find the valid schedule.

1. Initialize domains: $A \in \{1, 2, 3\}, B \in \{1, 2, 3\}, C \in \{1, 2, 3\}$.
2. Apply Constraint 2 ($C \neq 1$): Domain of C reduces to $\{2, 3\}$.
3. Apply Constraint 1 ($A < B$): A cannot be 3 (no room for B). B cannot be 1. Domains become $A \in \{1, 2\}, B \in \{2, 3\}$.
4. Apply Global Constraint (AllDifferent): Iteratively propagates domains to yield a conflict-free unique assignment for slots 1, 2, and 3.

6. Decomposition Methods

1. Exact Algorithms in Optimization
2. Complete Enumeration (Brute Force)
3. Dynamic Programming
4. Branch-and-Bound
5. Constraint Programming
6. **Decomposition Methods**

Motivation: Exploiting the **structural properties** of massive optimization problems by splitting them into **coordinated subsystems**.

- **Decomposition Methods:** Exploit special structure of large problems by splitting them into smaller, manageable subproblems coordinated iteratively through cuts or columns.
- **Structural Properties:** Requires block or nearly block diagonal structure in constraints/variables, or linking constraints that couple otherwise separable substructures.